

咖啡台 LED 控制 · TCP 自定义协议 v1.0

项	值
适用固件	coffe_mcu (ESP32-C5-N16R8)
版本	v1.0 (2026-09-15, 含健壮性/边界约定修订)
用途	客户/上位机作为 TCP 客户端主动连接设备, 通过自定义 JSON 协议控制 10 路 LED, 设备执行完毕后回一个应答报文。
说明	本协议用于对外控制; 另保留标准 Modbus TCP (端口 502) 作为调试/兼容接口 (见文末附录)。

1. 传输与连接

项	值
协议	TCP (面向连接)
监听端口	9001
连接方向	客户 (客户端) 主动 connect → 设备 (服务端) 应答; 连接后由客户发命令
并发连接	最多 4 个; 第 5 个连接被 accept 后立即 close (设备不保留、不回包)
心跳	客户可用 ping 命令测试连接; 设备侧空闲超时 60s (见 1.3)
帧格式	每条命令一个 JSON 对象 + 一个换行符 \n (LF, 0x0A); 设备每条应答同样是一个 JSON 对象 + \n
单条帧长度上限	512 字节 (含 \n)
文本编码	UTF-8 (含 msg 等中文文案)

1.1 帧长度与超长处理

- 设备按 \n 切分消息, 单条命令最长 512 字节 (含换行)。
- 若超过 512 字节: 设备丢弃该超长行, 回一条 INVALID_JSON (msg:"命令过长"), 不关闭连接, 继续处理后续命令。

1.2 TCP 粘包 / 半包处理

- 设备端按行缓冲: 收到数据先攒进缓冲区, 遇到 \n 才解析。
- 因此允许: 一条命令被拆分成多个 TCP 段发送 (半包); 多条命令一次性连续发送 (粘包)。
- 客户端无需特殊处理, 只需保证每条命令以 \n 结尾即可。

1.3 空闲超时 / 断连

- 设备端空闲超时：连接 60 秒内无任何数据，设备主动关闭该连接并释放槽位（防止异常掉线、无 FIN 的死连接占满 4 个并发槽位）。
- 客户端正常 `close`（含半关闭）设备也会立即处理并释放槽位。
- 建议客户端侧也设置连接超时并定期用 `ping` 保活/探活。

1.4 流水线（pipelining）

- 允许客户端连续发送多条命令而无需等待应答。
- 设备按接收到的先后顺序处理，并按相同顺序返回应答（每条命令固定对应一条应答，顺序一一对应）。
- 客户端可据此做批量下发；如需严格的请求-应答匹配，请给每条命令带唯一的 `seq`。

2. 消息通用结构

2.1 请求（客户 → 设备）

```
{ "seq": 123, "cmd": "led_set", "ch": 1, "on": true }
```

字段	类型	必填	说明
<code>seq</code>	int	否	客户自定序号；应答原样回传，用于请求-应答匹配。范围：0 ~ $2^{31}-1$ (int32，建议每次请求递增)
<code>cmd</code>	string	是	命令名（见第 4 节）
其余	—	按命令	各命令参数

2.2 应答（设备 → 客户，执行完毕后发送）

成功：

```
{ "seq": 123, "ok": true, "cmd": "led_set", "data": { "ch": 1, "on": true } }
```

失败：

```
{ "seq": 123, "ok": false, "cmd": "led_set", "err": "INVALID_CHANNEL", "msg": "ch 需为 1..10" }
```

字段	类型	说明
<code>seq</code>	int?	若请求带了 <code>seq</code> 则回传，否则应答不含 <code>seq</code> 字段（严格按此规则）
<code>ok</code>	bool	<code>true</code> 成功 / <code>false</code> 失败
<code>cmd</code>	string	对应命令名（无法解析时可能为 <code>""</code> ）
<code>data</code>	object?	成功时的业务数据，见各命令

err	string?	失败时的错误码，见第 5 节
msg	string?	失败时的中文说明（UTF-8）

3. LED 通道与类型约定

- 共 10 路：通道号 CH1 ~ CH10（取值 1..10）。
- 状态值严格为布尔：请求里 on 只接受 JSON 布尔 true / false；传 1 / 0、"true" / "false" 字符串均视为非法（返回 INVALID_VALUE）。
- 应答里 data.on 恒定输出 true / false（不输出 1 / 0）。
- 与 Modbus 寄存器对应关系见附录（Modbus 地址 0x0000..0x0009 = CH1..CH10）。

4. 命令集

4.1 ping — 连接测试 / 心跳

请求：{ "cmd": "ping" }

成功应答 data: { "pong": true }

4.2 led_set — 单路开/关

请求：{ "cmd": "led_set", "ch": 1, "on": true }

成功应答 data: { "ch": 1, "on": true }

错误：INVALID_CHANNEL（ch 不在 1..10）、INVALID_VALUE（on 非布尔 true / false）

4.3 led_set_all — 全开 / 全关

请求：{ "cmd": "led_set_all", "on": true }

成功应答 data: { "count": 10, "on": true }

说明：count 为实际生效路数；on 为本次统一设置的状态（布尔）。

错误：INVALID_VALUE（on 非布尔）

4.4 led_set_batch — 多条一起设置（一次应答）

请求：{ "cmd": "led_set_batch", "list": [{"ch":1,"on":true}, {"ch":3,"on":false}] }

成功应答 data: { "applied": 2 }

说明：先整体校验再统一执行；任一条非法则整条返回错误、不执行任何一条。applied = 成功设置的条数。

错误：INVALID_CHANNEL、INVALID_VALUE、INVALID_LIST（list 缺失/非数组/元素非法）

4.5 led_status — 读当前全部状态

请求：{ "cmd": "led_status" }

成功应答 data:

```
{ "channels": [ {"ch":1,"on":true}, {"ch":2,"on":false}, ... ] }
```

说明: `channels` 固定为 10 项, 顺序即 CH1..CH10; `on` 恒为布尔。

5. 错误码

err	说明
INVALID_JSON	JSON 无法解析 / 空串 / 命令超长
INVALID_CMD	<code>cmd</code> 缺失或未知
INVALID_CHANNEL	<code>ch</code> 不在 1..10
INVALID_VALUE	<code>on</code> 不是布尔 <code>true</code> / <code>false</code> , 或数值非法
INVALID_LIST	<code>list</code> 缺失或不是数组 / 数组元素格式非法
INTERNAL	设备内部异常 (如 NVS 写失败)

6. 交互示例

以下 `->` 行为客户端发送, `<-` 行为设备应答。客户 `connect 192.168.1.100:9001`。

点亮 CH3 (带 seq, 回传匹配)

```
-> {"seq":1,"cmd":"led_set","ch":3,"on":true}
<- {"seq":1,"ok":true,"cmd":"led_set","data":{"ch":3,"on":true}}
```

读全部状态 (请求不带 seq, 则应答也不含 seq)

```
-> {"cmd":"led_status"}
<- {"ok":true,"cmd":"led_status","data":{"channels":[{"ch":1,"on":true},{"ch":2,"on":false},
{"ch":3,"on":true},{"ch":4,"on":false},{"ch":5,"on":false},{"ch":6,"on":false},{"ch":7,"on":false},
{"ch":8,"on":false},{"ch":9,"on":false},{"ch":10,"on":false}]}}
```

非法通道

```
-> {"seq":2,"cmd":"led_set","ch":11,"on":true}
<- {"seq":2,"ok":false,"cmd":"led_set","err":"INVALID_CHANNEL","msg":"ch 需为 1..10"}
```

粘包示例 (一次发两条, 按序回两条应答)

```
-> {"cmd":"led_set","ch":1,"on":true}{"cmd":"led_set","ch":2,"on":true}
<- {"ok":true,"cmd":"led_set","data":{"ch":1,"on":true}}
<- {"ok":true,"cmd":"led_set","data":{"ch":2,"on":true}}
```

7. 安全说明

- 本协议 v1.0 无鉴权/令牌: 连上即可控制全部 LED, 且设备入网后 9001 常驻监听。请部署于可信局域网 / VLAN, 或用交换机/防火墙做网段隔离。

- 若交付到客户现场网络（非完全可信内网），强烈建议启用 v1.1 的连接期访问令牌（改动很小）。启用后，连接建立后需先发一条鉴权命令（返回 `ok`）才能执行其它命令。
- 其余 HTTP 配网/状态接口同属无鉴权，请一并纳入网络防护考虑。

附录 A · 与 Modbus TCP 从站（端口 502，调试用）

设备同时在 端口 502 提供标准 Modbus TCP 从站（用于调试/兼容）：

Modbus 项	值
功能码	<code>0x03</code> 读保持寄存器 / <code>0x06</code> 写单个 / <code>0x10</code> 写多个
寄存器地址	<code>0x0000..0x0009</code> = CH1..CH10
寄存器值	<code>0</code> =灭, <code>1</code> =亮
工具	mbpoll / Modbus Poll / pymodbus

并发写注意：Modbus 从站与自定义 TCP 协议（9001）可同时监听、互不干扰（都只占网络端口、不占串口）。但两者若并发写同一通道，其语义是“最后到达者生效”（last-write-wins），无原子组合语义（批量也无事务保证）。请在调试/现场避免两套接口同时操作同一通道；确需同时用，请上位机侧自行加互斥。